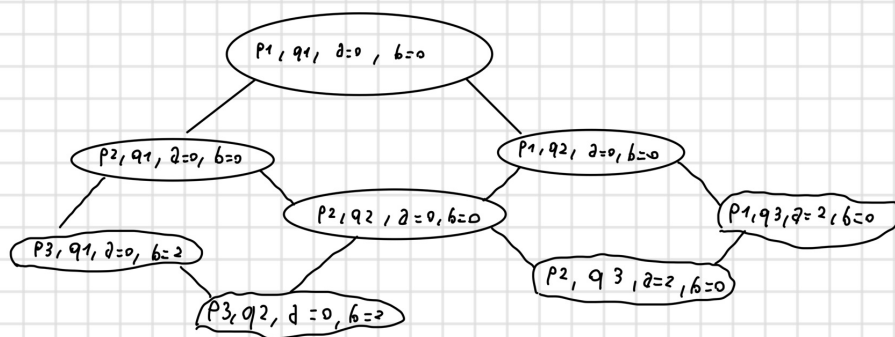


1)



2) SI LA VERIFICA. SUPPONIAMO CHE L'ALGORITMO NON VERIFICHI MUTUA ESCLUSIVITA', QUESTO SIGNIFICA CHE ALLO STESSO TEMPO LE DUE VARIABILI CONTINUTE d E b DOVRANNO ESSERE ALTERNAMENTE $= 0$, MA QUESTO NON E' POSSIBILE IN QUANTO PERCHÉ DI q_5 E p_5 p O q SONO PASSATI PER p_2 O q_2 CHE (MANTENENDO) $b=2$ O $d=2$ QUINDI $\neq 0$

3) NO, IN QUANTO SE LO SCHEMA DOVESSE ESEGUIRE IN PARALLELO ENTRAMBI I PROCESSI, ENTRAMBI NIENTAMENTE BLOCCATI NEI DUE CICLI WHILE.

4) LA PROPRIETA' DI MUTUA ESCLUSIVITA' NON E' VERIFICATA IN QUANTO SE LO SCHEMA DOVESSE ESEGUIRE PRIMA Q E POI P ENTRAMBI ANDREBBERO IN JC.

		d	b
P_1	q_1	0	0
"	q_2	0	0
"	q_3	2	0
"	q_5	2	0
P_2	"	"	"
P_3	"	"	2
P_4	"	"	2
P_5	"	0	"

Esercizio 2

// VAR CONDIZIONE

LOCK L

INT N_CCOND = 0

BOOL PENTE_ABBATE : TRUE FLAG PER CAPIRE SE L'ALLEGGERE E' PERNO AL PERNO

VAR COND ANTELA_JANNA // AZIONE SECONDO UGUALI PER JUDGE

VAR COND ANTELA_DUEJA // AZIONE SECONDO UGUALI PER JUDGE

VAR COND ANTELA_PENTE_ABBATE

VAR COND ANTELA_VOTO

// UGUALI

WHILE (1) {

LOCK (L)

WHILE (M_PENJONE >= 5 || ! PENTE_ABBATE) { SE L'ALLEGGERE E' PERNO O E' IN VACCIA, LO UGUALI DEVO' ASPETTARE

WAIT (ANTELA_JANNA, L)

M_PENJONE++

SIGNAL (ANTELA_PENTE_ABBATE, L) AZIONE L'ALLEGGERE DI PENTE

UNLOCK (L)

// VACCIA

LOCK (L)

WHILE (! PENTE_ABBATE) {

WAIT (ANTELA_DUEJA, L)

M_PENJONE--

IF (M_PENJONE == 0) {

SIGNAL (ANTELA_VOTO, L)

}

UNLOCK (L)

}

// ASSEGNAZIONE

```

WARE (1) {
    LOCK(L)
    WHILE (M_PERSONE == 0) {
        WAIT (AX - PASTI - 20, L)
    }
    PONE_APERTA = FALSE
    UNLOCK(L)

    // VIACCIA

    // ARRIVA
    LOCK(L)
    PONE_APERTA = TRUE
    BROADCAST (ANTELA_DUELA, L) // VERBA TUM I PASTI
    // ANTELA DUELA VUOTO
    WHILE (M_PERSONE > 0) {
        WAIT (AX - VUOTO, L)
    }
    // ANTELA VUOTO, FA JOLIE CUI AJTAI
    BROADCAST (ANTELA_JOLIE, L)
    UNLOCK(L)
}

```

Esercizio 3

- 1) Quando un processo P_i vuole entrare in SC , segnala questo stato impostando la sua variabile locale $CS = TRUE$. P_i entra in ciclo $WARE$. All'interno del ciclo continua a inviare messaggi di richiesta agli altri due processi del sistema, aspettando che la somma tra $ACK1$ e $ACK2$ arrivi a 2. Il blocco in basso gestisce i messaggi di backchannel. Il controllo fondamentale è $IF(!CS)$ in questo manca un GO solo se non sta cercando di entrare a sua volta in SC . Se riceve il GO allora imposta a 1 $ACK1$ e $ACK2$. Una volta arrivati i messaggi P_i entra in SC .
- 2) Il algoritmo verifica la proprietà di mutua esclusione. Supponiamo se per assurdo non fosse vera:
 - 1) Supponiamo che due processi, ad esempio P_0 e P_1 siano in SC contemporaneamente
 - 2) Per tornare in SC , entrambi devono aver precedentemente impostato la propria variabile locale $CS = TRUE$
 - 3) Per superare il ciclo $WARE$ ed entrare in SC , P_0 ha bisogno di ricevere il messaggio GO da P_1 e da P_2 . Allo stesso modo P_1 ha bisogno del GO da P_0
 - 4) Ma siccome il controllo arriva un GO solo se $IF(!CS)$ è vera, se entrambi hanno $CS = TRUE$ nessun inviato il GO all'altro.
- 3) No, l'algoritmo non garantisce l'assenza di dead lock. Se tutti i processi tentano di entrare in SC nello stesso istante, impostano tutti contemporaneamente $CS = TRUE$. Quando i processi ricevono ACK , il controllo arriva al punto GO per tutti. Ovvero nessuno manda il GO e tutti i processi saranno bloccati per sempre nel $WARE$.
- 4) Deadlock $\Rightarrow$ starvation.